

Patrones de agentes y diseño del toolset

MMIA 6013 — IA Generativa y Agentes · Sesión 12 · martes 29 de septiembre de 2026

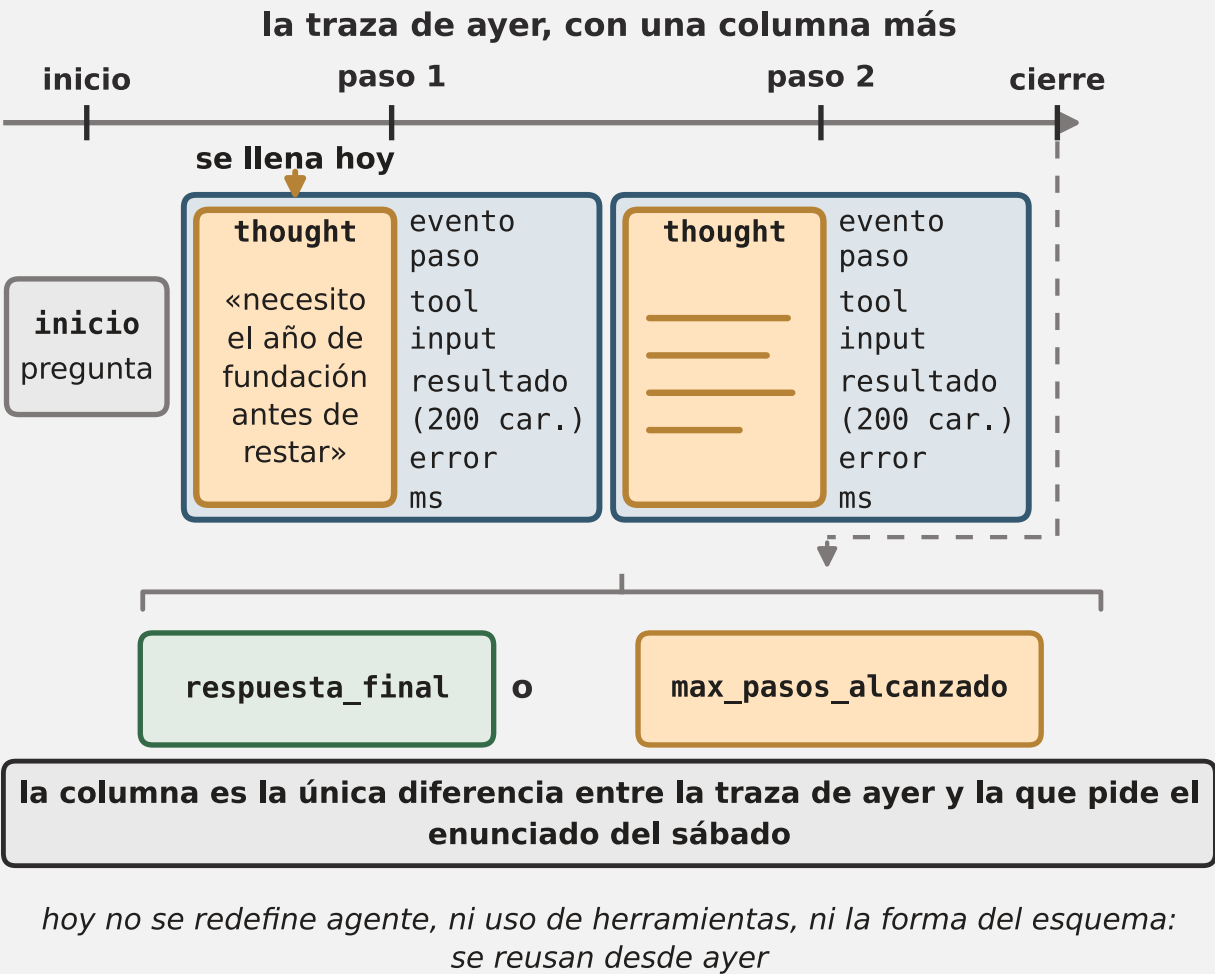
Semana 3 · RA-4 · 90 minutos de deck y discusión, 20 de autopsia de trazas en grupos

Segunda sesión de la semana de agentes. El Taller 3 es el sábado 3 de octubre

Daniel Andrés Riofrío Almeida

La columna que ayer quedó vacía

- Ayer construyeron el bucle y su traza: herramienta, entrada, resultado, error y cuánto tardó (ms, milisegundos)
- Esa traza tenía una columna tramada y vacía, rotulada *thought*
- El Taller 3 pide registrar Thought, Action y Observation: ese vocabulario es de hoy
- Hoy, primera mitad: tres patrones que dicen QUÉ decide el bucle en cada paso
- Segunda mitad: el contrato de la herramienta que ese bucle invoca, y sus errores
- No se redefine agente, ni function calling, ni la forma del esquema: se reusan desde aquí



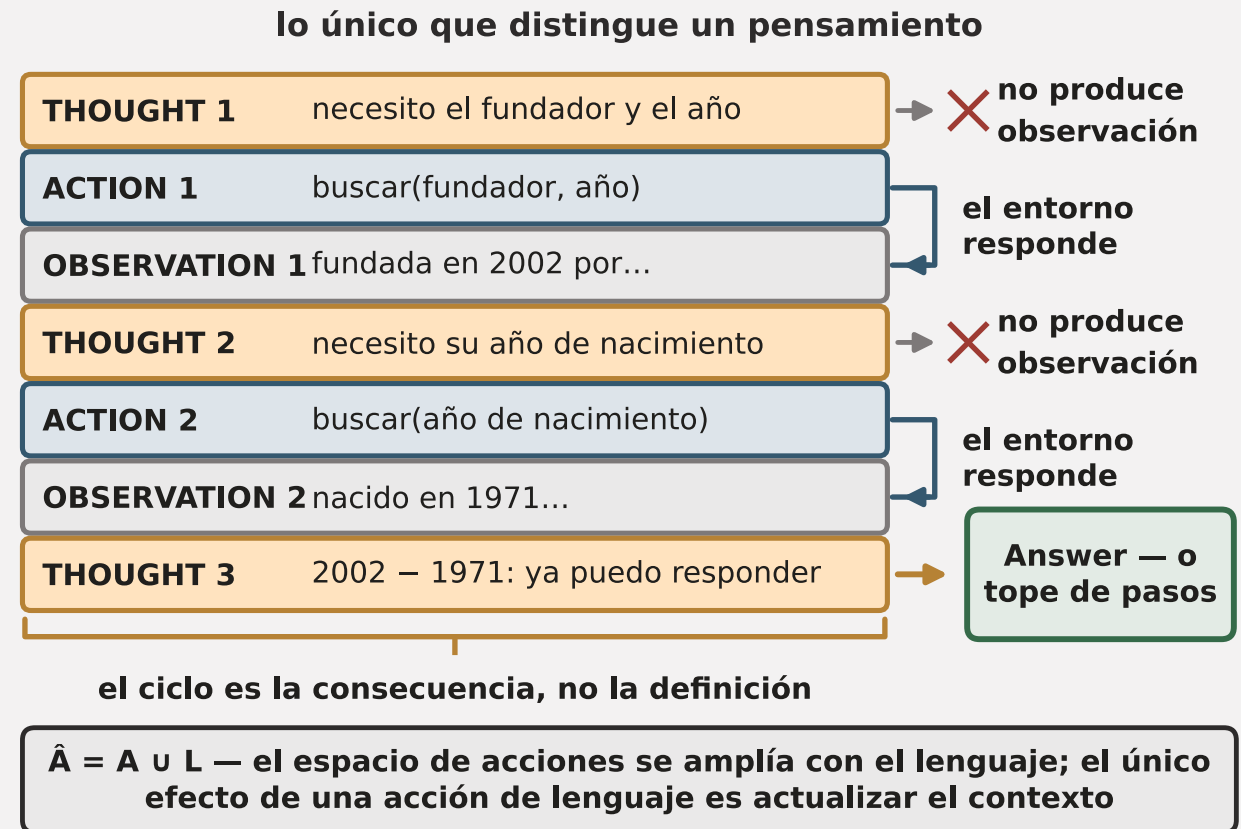
Fuente: [deprecado/slides/sesion-11/outline.yaml](#), d22 (la traza con la columna *thought* vacía) y d15 («la descripción es el prompt»); [curso/talleres/taller-03-agentes.md](#), línea 46 (el trace registra cada Thought, Action y Observation); [slides_v2/sesion-12/contenido-base.md](#) §1.2

ReAct: un pensamiento es una acción

12

Un pensamiento es una acción más, con un solo efecto

- A : las acciones sobre el entorno (buscar, consultar). ReAct las amplía: $\hat{A} = A \cup L$, con L el espacio del lenguaje
- Una acción de lenguaje —un pensamiento, $\hat{a}_t \in L$ — NO afecta al entorno y por tanto NO produce observación
- Su único efecto es alargar el contexto c_t , la historia de observaciones y acciones hasta el paso t : $c_{t+1} = (c_t, \hat{a}_t)$
- Eso es toda la definición. El ciclo pensamiento $\rightarrow$ acción $\rightarrow$ observación es la consecuencia
- Tipos de pensamiento que el paper observó: descomponer el objetivo, inyectar sentido común...
- ...extraer lo importante de una observación, seguir el progreso y ajustar el plan de acción



traza redibujada a partir de Yao et al. 2023, §2, p. 3; ninguna figura del paper se incrusta

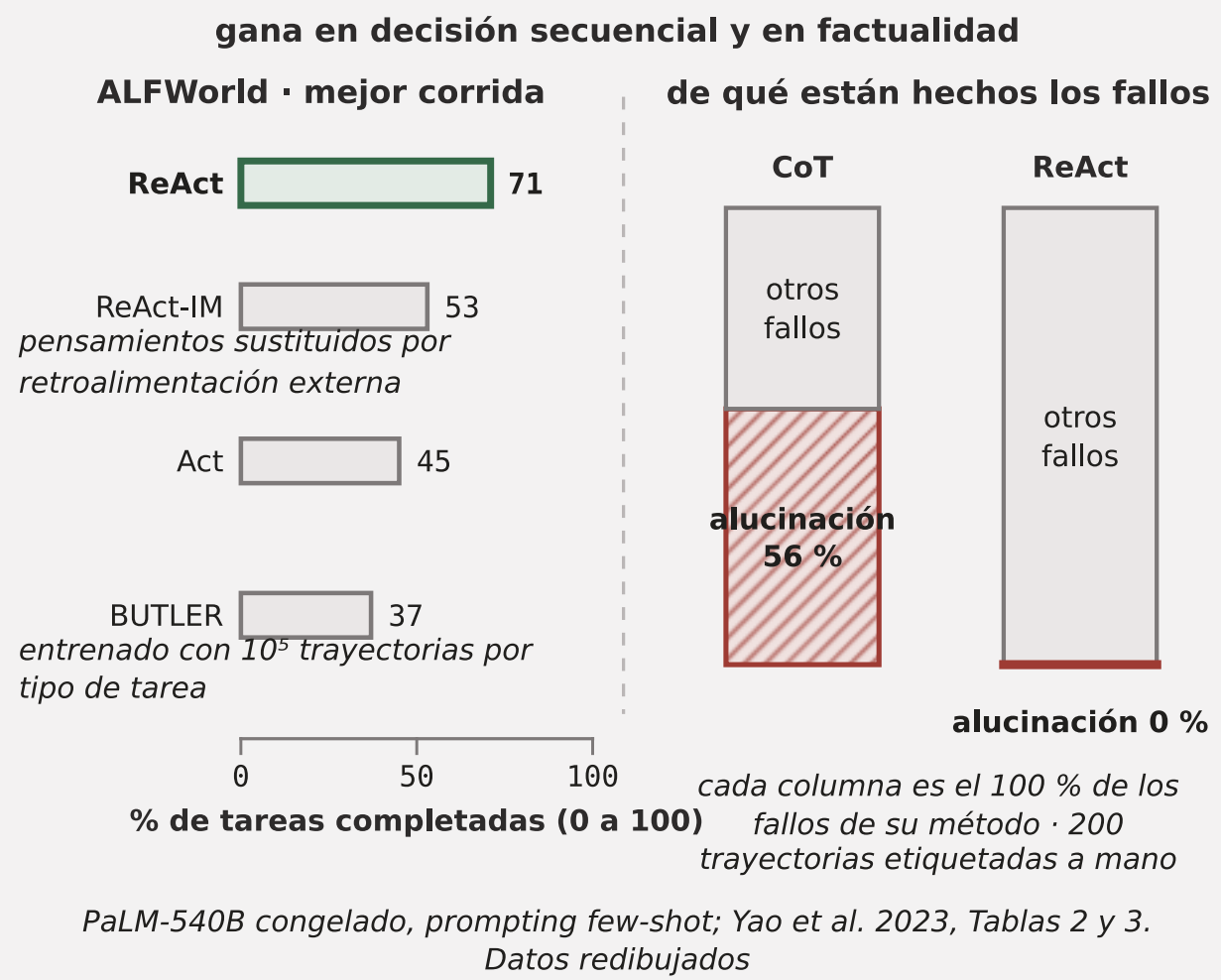
De dónde sale ReAct, qué probó y qué nos deja

- Yao et al., de Princeton y Google Research (Brain): arXiv en octubre de 2022, publicado en ICLR 2023
- El hueco que atacan: razonar (chain-of-thought) y actuar (generar acciones) se estudiaban por separado
- La idea: intercalarlos. El pensamiento decide la acción; la acción trae del entorno lo que el modelo no sabe
- Lo probaron en cuatro tareas: HotpotQA y FEVER contra una API de Wikipedia; ALFWorld y WebShop, de decisión
- Frente a qué: solo razonar (CoT), solo actuar (Act) y agentes entrenados por imitación o por refuerzo
- Qué nos deja: una traza auditable —qué sabía el modelo y qué vino de fuera— y el vocabulario del Taller 3

Fuente: Yao et al. (2023), ReAct, p. 1 (autores, filiaciones, arXiv:2210.03629, las cuatro tareas y el resumen: razonar y actuar «estudiados como temas separados»); §1, p. 2 (CoT como «caja negra estática» que no toca el mundo, frente a generación de acciones) y p. 3 (interpretabilidad de la traza); curso/talleres/taller-03-agentes.md, línea 46 (el trace registra cada Thought, Action y Observation)

Dónde ReAct gana, y con qué modelo está medido

- Montaje: PaLM-540B congelado y few-shot en contexto — 6 ejemplos en HotpotQA, 3 en FEVER
- Espacio de acciones de las tareas de QA: solo tres — search, lookup, finish
- ALFWorld, mejor corrida: ReAct 71 % de éxito · Act 45 % · BUTLER 37 %
- BUTLER es imitación entrenada con 10^5 trayectorias por tipo de tarea; ReAct usa 1 o 2
- 200 trayectorias etiquetadas a mano: la alucinación es el 56 % de los fallos de CoT y el 0 % de los de ReAct
- Ablación: sustituir los pensamientos por retroalimentación externa densa baja de 71 a 53



Fuente: Yao et al. (2023), §2, p. 3 (PaLM-540B congelado, few-shot); §3.1, p. 4 (las tres acciones); Tabla 3 y §4, p. 8 (ALFWorld, BUTLER y la ablación ReAct-IM); Tabla 2, p. 6 (etiquetado humano de 200 trayectorias). Datos redibujados; ninguna figura se incrusta

«ReAct mejora el desempeño.» ¿En qué, exactamente?

Misma tabla principal del paper,
mismo modelo base y mismo régimen
de prompting

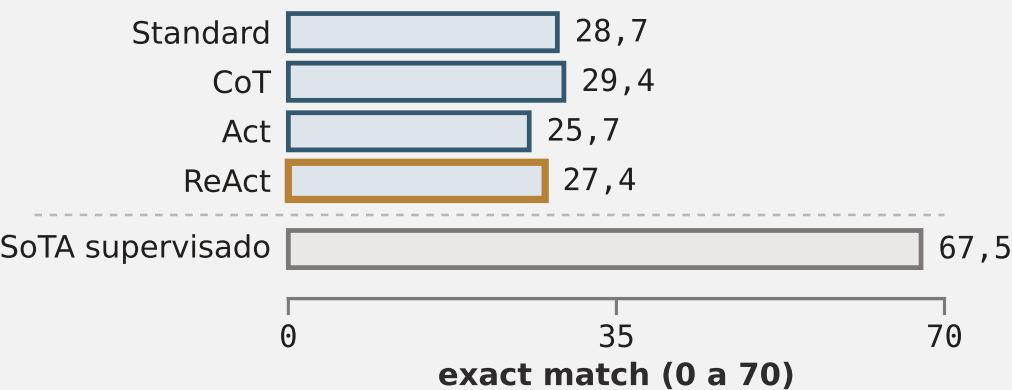
HotpotQA (exact match) — Standard
28,7 · CoT 29,4 · Act 25,7 · ReAct 27,4

FEVER (accuracy) — Standard 57,1 ·
CoT 56,3 · Act 58,9 · ReAct 60,9

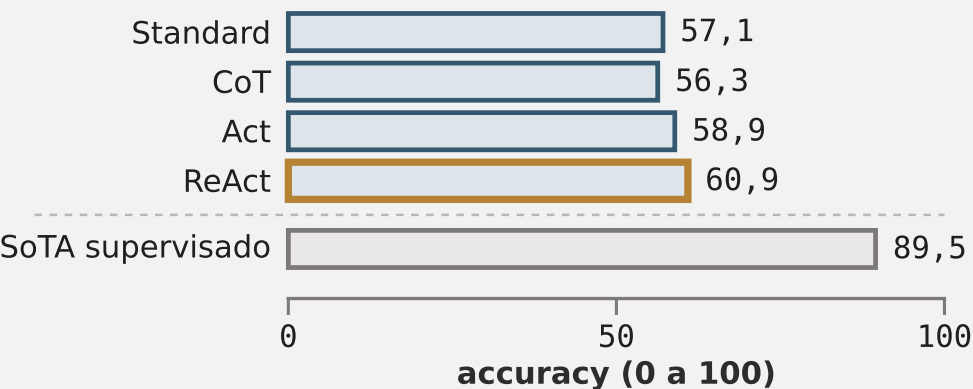
SoTA supervisado del momento —
67,5 en HotpotQA y 89,5 en FEVER

(a) ¿ReAct ganó? (b) ¿Contra qué línea
base lo dirían en una reunión de
trabajo?

la misma técnica ordena distinto según el benchmark
HotpotQA · exact match



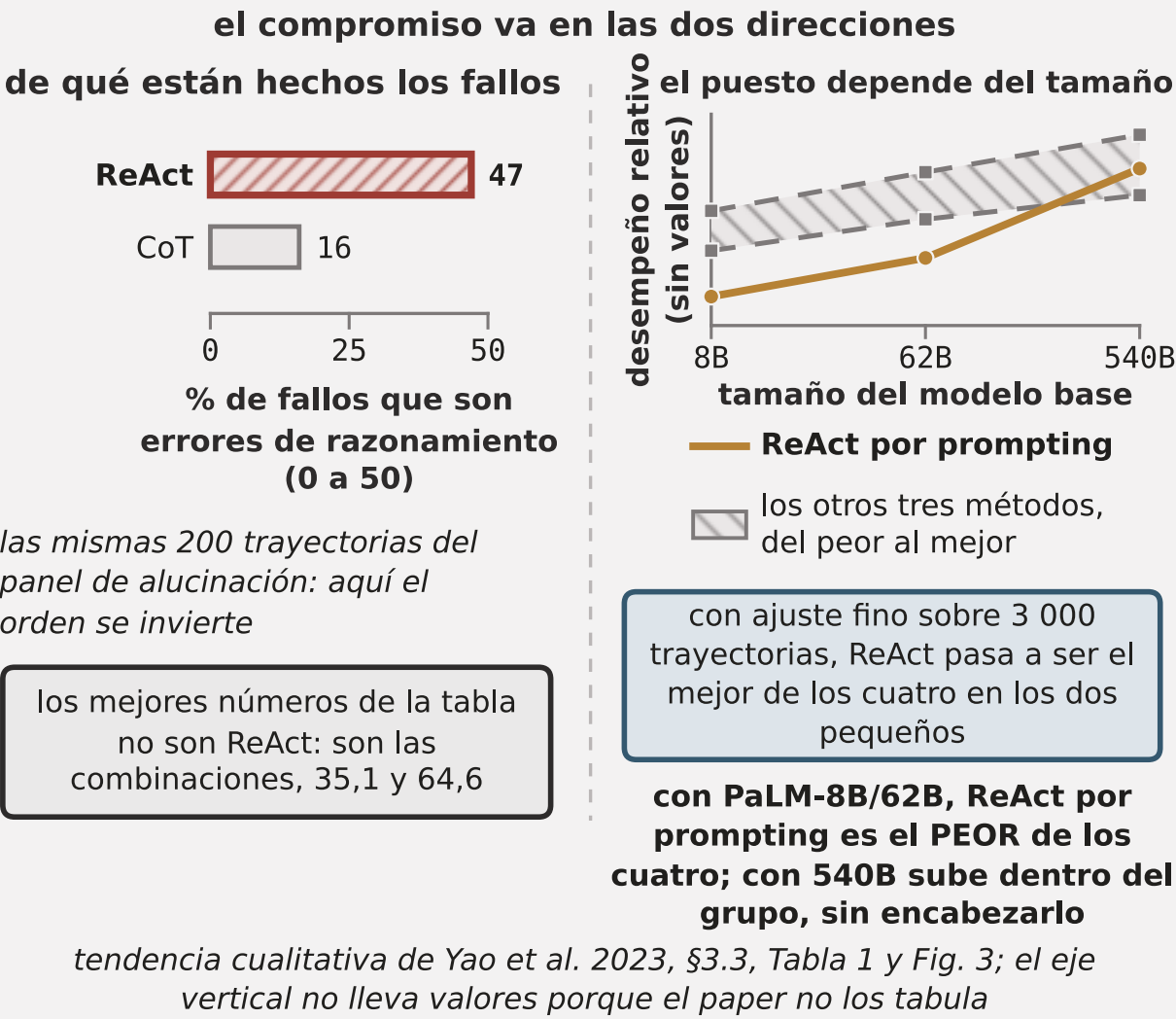
FEVER · accuracy



PaLM-540B congelado; los cuatro métodos comparten modelo y régimen
de prompting. Yao et al. 2023, Tabla 1, p. 5

Lo que la tabla explica, y por qué no basta con elegir el patrón

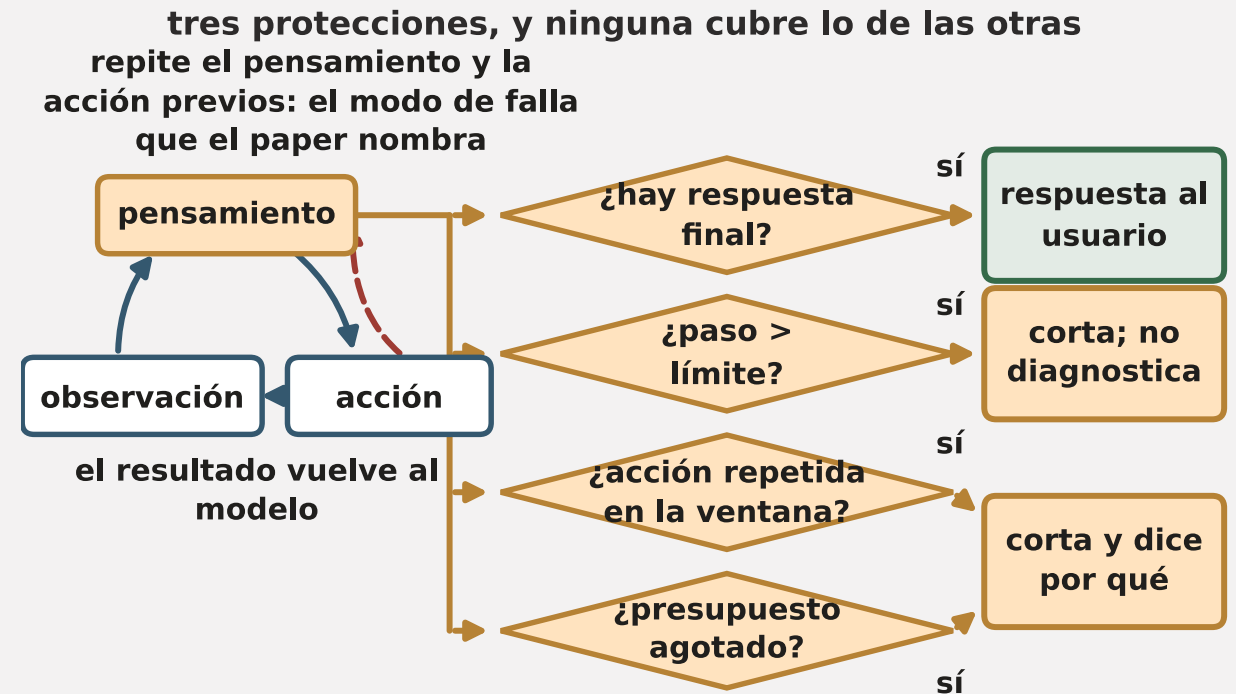
- El paper lo llama un compromiso esperado: intercalar reduce la flexibilidad de razonar
- Sobre las mismas 200 trayectorias: 47 % de errores de razonamiento en ReAct, 16 % en CoT
- Lo que ReAct compra en factualidad lo paga en flexibilidad. Va en las dos direcciones
- Los mejores números de la tabla no son ReAct: son las combinaciones, 35,1 y 64,6
- La conmutación es una regla explícita: si ReAct no responde en sus pasos, se cae a CoT-SC
- Con PaLM-8B/62B, ReAct por prompting es el PEOR de los cuatro; con ajuste fino, el mejor



Fuente: Yao et al. (2023), §3.3, p. 6 y Fig. 3 (el compromiso, el 47 % frente al 16 %, y la inversión con modelos pequeños); Tabla 1 y §3.2, p. 5 (las combinaciones y su heurística de conmutación); §6, pp. 9-10

Cuando el agente se repite: la falla y sus tres cortes

- La falla: el agente repite el mismo pensamiento y la misma acción, y nunca responde
- El paper de ReAct la documenta como su error frecuente, y sospecha del decodificado greedy
- Pedirlo en el prompt no basta: la plantilla del libro dice «can repeat N times» y no pone tope en el código
- Corte 1, límite de pasos: siempre termina, pero no dice qué falló. Solo el 0,84 % y el 1,33 % de los aciertos llegan al tope
- Corte 2, detector de repetición: dice por qué cortó, pero no ve la misma búsqueda con otras palabras
- Corte 3, presupuesto: frena el gasto, no a un agente barato y equivocado. Hacen falta los tres



lo que cada protección NO atrapa

el límite de pasos corta siempre, pero no diagnostica nada
el detector de repetición (ventana de 6) no atrapa la paráfrasis
el presupuesto corta el gasto, no la calidad

*viven en el código del bucle, no en el prompt · el 0,84 % y el 1,33 %
de las trayectorias correctas consumen 7 y 5 pasos*

Fuente: Yao et al. (2023), §3.3 y nota 4, p. 6 (la repetición y el decodificado greedy); §3.2 y nota 3, p. 5 (7 y 5 pasos; 0,84 % y 1,33 %); Alammar & Grootendorst (2024), cap. 7, §«ReAct in LangChain», pp. 221–222 (la plantilla lleva «can repeat N times» dentro del prompt y su ejecutor se construye sin tope de iteraciones); curso/actividades/semana-3/s3-mar-explicacion.html, línea 144 (la protección vive en el código del loop) y _fuentes/s3-mar.py, líneas 97–108 y 115–117

Autopsia de tres trazas — 20 minutos, en grupos

- Cada grupo recibe tres trazas ReAct impresas; las mismas están en el notebook del día
- Una está sana. Una tiene un bucle: repite la misma búsqueda con palabras distintas
- Una tiene una fabricación: un Thought afirma algo que ninguna Observation respalda
- 1 · Diagnostiquen cada una: ¿sana o enferma?, y ¿en qué línea exactamente se rompe?
- 2 · Propongan la intervención: ¿límite de pasos? ¿detector de repetición? ¿mejor descripción de la tool? ¿verificador?
- Las Partes 2 a 4 del notebook se trabajan sobre estas trazas y se terminan fuera de clase

tres trazas, sin etiqueta

P pensamiento · A acción · O observación

A

P

A

O

P

A

O

B

P

A

O

P

A

O

C

P

A

O

P

A

O

el menú de intervenciones, sin asignar

- límite de pasos
- mejor descripción de la tool

- detector de repetición
- verificador

1 · diagnostiquen cada traza y digan en qué línea se rompe. 2 · propongan la intervención

el diagrama es el enunciado del ejercicio: no dice cuál es cuál

Fuente: [curso/actividades/semana-3/s3-mar-explicacion.html](#), líneas 95–106 (la actividad, sus tres trazas y sus 20 minutos, literales); [curso/actividades/_fuentes/s3-mar.py](#), líneas 91–117 (detectar_loop, su normalización y su límite ante la paráfrasis). Que el notebook se termine fuera de clase es [AGREGADO]

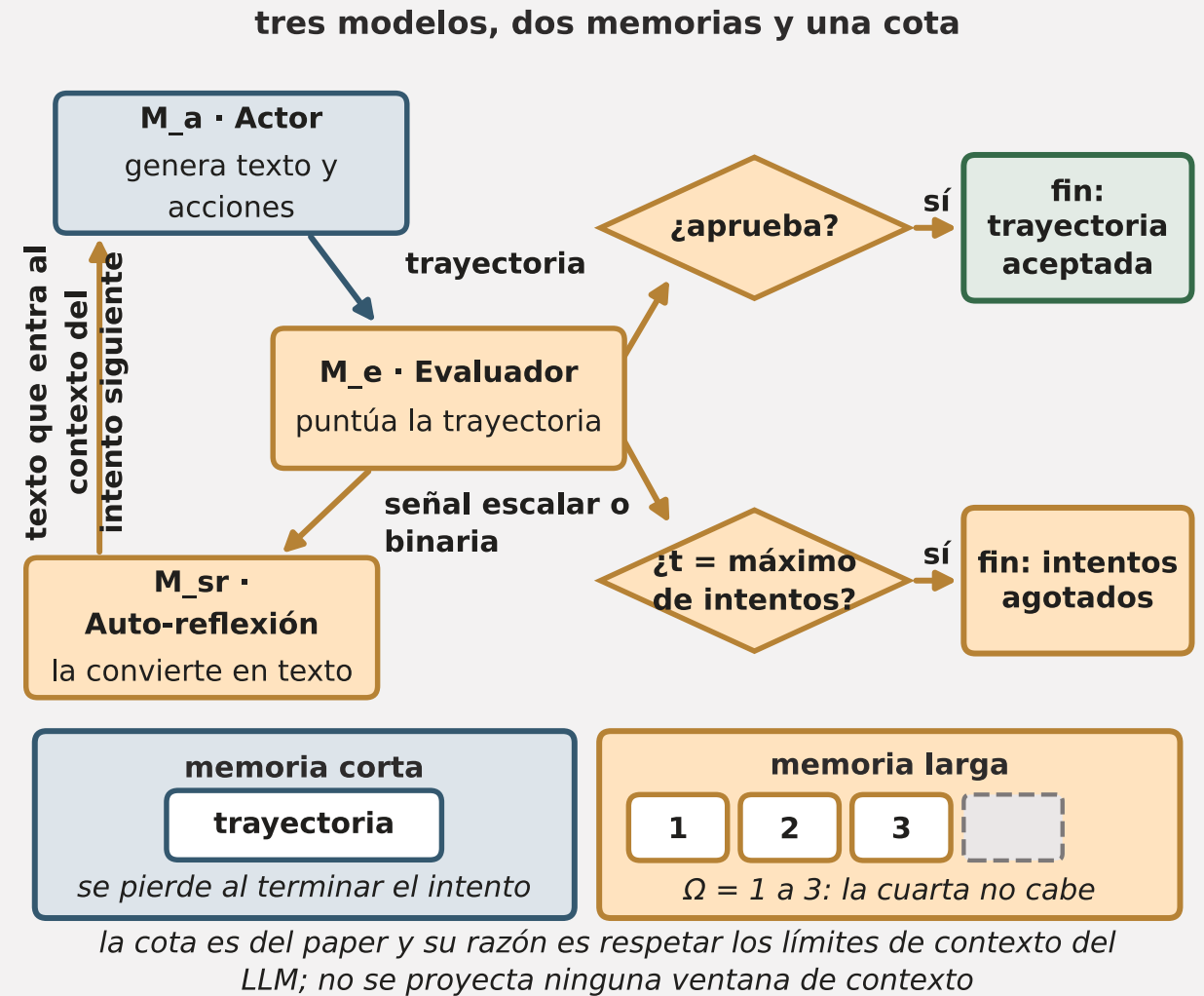
10

Reflexion: el patrón entre intentos

12

Reflexion no es un prompt: son tres modelos y dos memorias

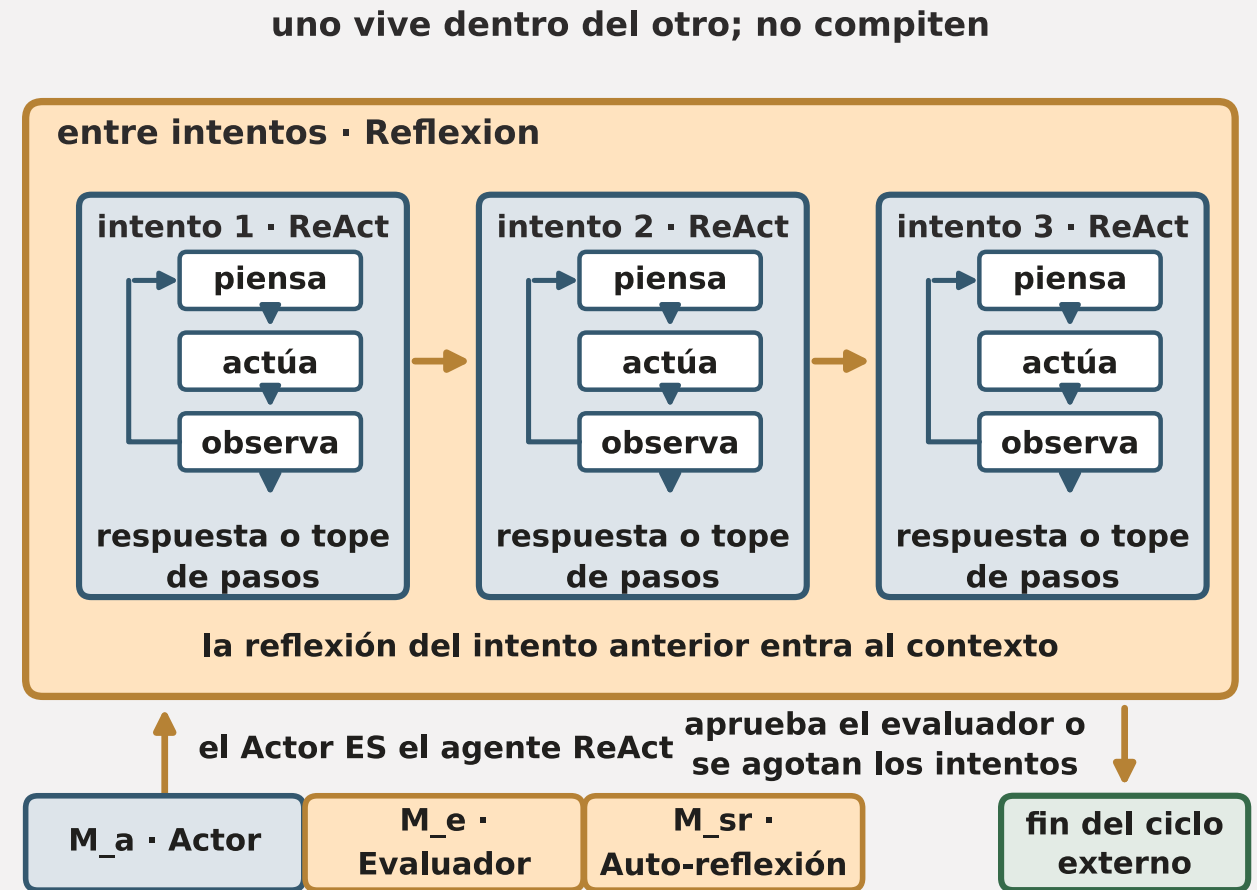
- Actor M_a genera texto y acciones —«incluidos Chain of Thought y ReAct»—; Evaluador M_e puntúa la trayectoria que produjo
- Auto-reflexión M_{sr} : convierte la señal del evaluador en TEXTO para el contexto del intento siguiente
- Memoria corta = la trayectoria del intento. Memoria larga = las auto-reflexiones acumuladas
- La larga se acota a Ω , «normalmente entre 1 y 3», «para respetar los límites de contexto del LLM»
- Acotar una memoria tiene tres formas —guardarlo todo, las últimas k , o resumir—: Reflexion usa la segunda, con $k = 3$
- El bucle para cuando el evaluador aprueba O se agotan los intentos: la condición está en el algoritmo



Fuente: Shinn et al. (2023), Reflexion, §3, pp. 3–4 (los tres modelos, las dos memorias, la cota Ω entre 1 y 3) y Algoritmo 1, p. 4 (la condición de parada); §4.1, p. 5 («truncamos la memoria del agente a las últimas 3 auto-reflexiones»). Preprint bajo revisión. Las tres formas de acotar y su coste: Alammar & Grootendorst (2024), cap. 7, §«Conversation Buffer» p. 210, §«Windowed Conversation Buffer» p. 212, §«Conversation Summary» pp. 214–217 (Tabla 7-1, parafraseada; no se transcribe)

ReAct y Reflexion no compiten: uno vive dentro del otro

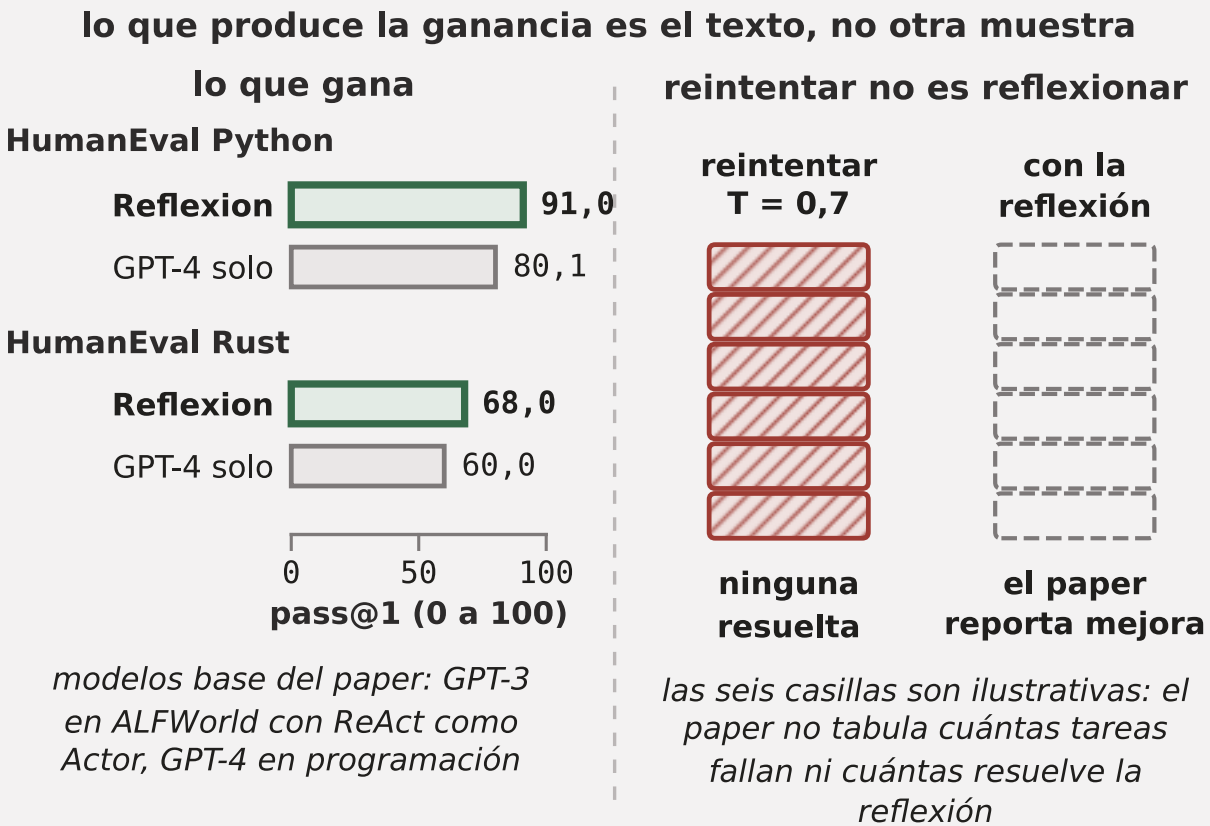
- Error de nivel frecuente: ponerlos en la misma lista y elegir uno de los dos
- El paper de Reflexion corre literalmente «ReAct + Reflexion» en ALFWorld
- Su Actor puede ser Chain of Thought o ReAct: Reflexion no reemplaza el patrón del bucle, lo envuelve
- ReAct es el patrón DENTRO de un intento. Reflexion es el patrón ENTRE intentos
- Se componen; no se eligen. La pregunta correcta no es cuál, es si hay más de un intento
- Consecuencia de diseño: si su bucle no deja traza, no hay nada sobre lo que reflexionar



*en Shinn et al. 2023 la corrida de ALFWorld se llama «ReAct + Reflexion»:
se componen, no se eligen*

Lo que Reflexion gana, y el resultado que de verdad lo justifica

- ALFWorld, GPT-3 con ReAct como Actor: 130 de 134 tareas; el enfoque de solo ReAct se estanca entre el intento 6 y el 7
- Programación con GPT-4: HumanEval Python 91,0 frente a 80,1 del modelo base; Rust 68,0 frente a 60,0
- HotpotQA, 100 preguntas: 20 % de mejora sobre el mejor comparador
- Y el dato que vale más: reintentar con $T = 0,7$ NO resolvió ninguna tarea previamente fallada
- Reintentar no es reflexionar. Lo que cambia el resultado es el texto en el contexto, no otra muestra
- La señal siempre viene de fuera: entorno binario, heurística escrita a mano, o tests que el agente genera



la señal siempre viene de fuera: entorno binario, heurística escrita a mano, o tests que el agente genera

Fuente: Shinn et al. (2023), §4.1, p. 5 (ALFWorld, GPT-3, ReAct como generador de acciones, y el estancamiento de solo-ReAct); Tabla 1, p. 7 (HumanEval Python y Rust, GPT-4); §4.2, p. 6 (HotpotQA y el resultado de $T = 0,7$); §1, p. 2 (las tres fuentes de señal). Datos redibujados

«Que el agente revise su propia respuesta solo puede mejorarla.»

Ablación del propio paper:
HumanEval Rust, los 50 problemas más difíciles, GPT-4, pass@1

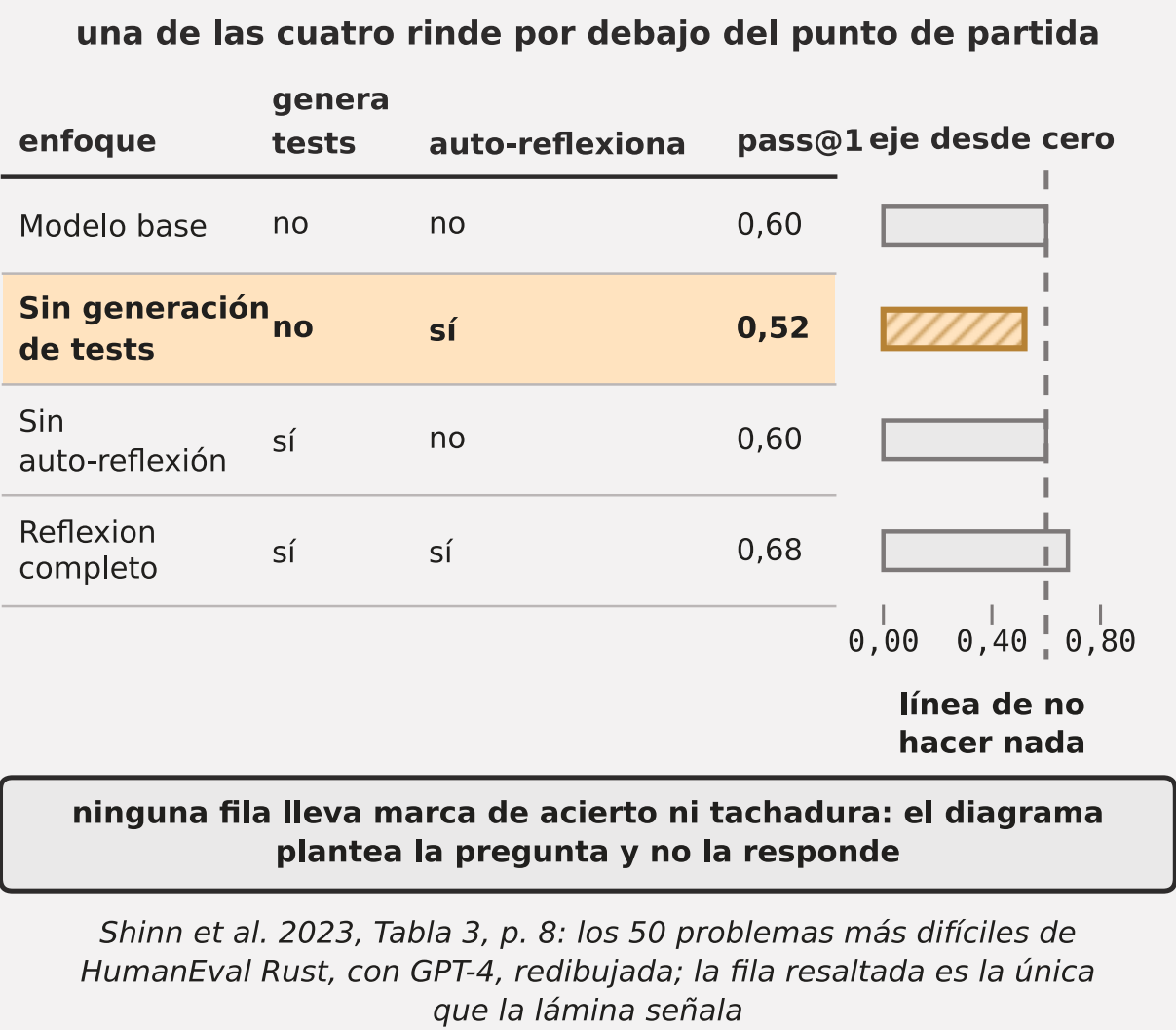
Modelo base — sin generar tests, sin auto-reflexión: 0,60

Con tests, sin auto-reflexión: 0,60

Sin tests, con auto-reflexión: 0,52

Con tests y con auto-reflexión: 0,68

¿Por qué la tercera fila queda por DEBAJO de no hacer nada?



Reflexion no es mejor que su verificador

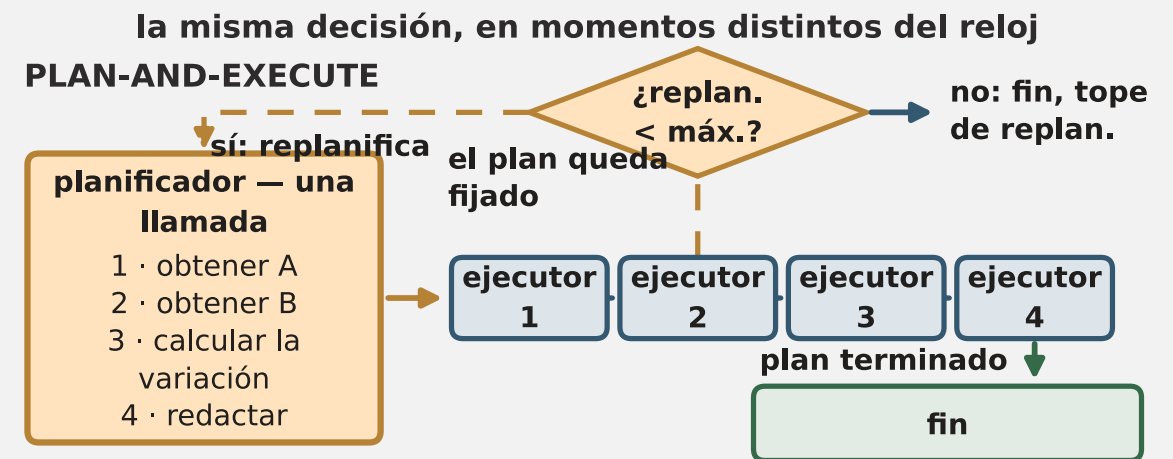
- Sin tests, la auto-reflexión resta 8 puntos: el agente no puede saber que ya había acertado
- El paper: «debe participar en todas las iteraciones sin la opción de retornar temprano, haciendo ediciones dañinas»
- Y pierde contra su propio modelo base en MBPP Python: 77,1 frente a 80,1
- La causa está medida: 16,3 % de falsos positivos de los tests en MBPP, frente a 1,4 % en HumanEval
- Un test que pasa sobre una implementación incorrecta hace que el agente entregue antes de tiempo
- Sus propios límites: mínimos locales, memoria de ventana deslizando, y «ninguna garantía formal de éxito»

El tercer patrón, y el aviso que lo acompaña

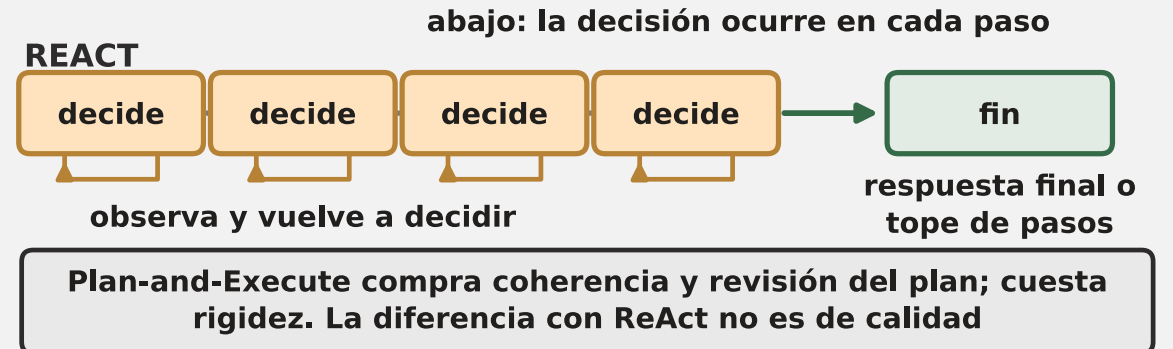
12

Plan-and-Execute: primero el plan entero, después la ejecución

- Aviso de evidencia: no está en los 23 papers del repositorio ni en el cap. 7 del libro del sílabo. Hoy se enseña el mecanismo, no un resultado
- Fase 1, planificador — una llamada, modelo capaz: genera el plan detallado de pasos ANTES de ejecutar
- Fase 2, ejecutor — n llamadas, puede ser un modelo barato: ejecuta cada paso «sin re-decidir la estrategia en cada turno»
- Compra: coherencia en tareas largas, aprobar el plan antes de ejecutarlo, y paralelizar pasos independientes
- Cuesta rigidez: replanificar lo mitiga y abre un segundo bucle. Sin un máximo de replanificaciones, la corrida no termina
- Tres puntos del mismo eje: en una cadena fija decidió el programador; aquí, el modelo una vez; en ReAct, en cada paso



arriba: la decisión ocurre una vez, antes



Plan-and-Execute compra coherencia y revisión del plan; cuesta rigidez. La diferencia con ReAct no es de calidad

sin fuente primaria en el repositorio: se enseña el mecanismo, sin cifras y sin comparación medida

Fuente: [fuentes/notas-teoricas/s3-agentes.md](#) §3.2, líneas 108–127 (las dos fases y su limitación); [curso/actividades/semana-3/s3-mar-explicacion.html](#) §2, líneas 64–74 (planner/executor, qué compra y qué cuesta). Sin fuente primaria en [fuentes/papers/](#): bloqueo B-1 del contenido.md. Los dos extremos del eje: Alammar & Grootendorst (2024), cap. 7, §«Chains», pp. 202–209, y §«Agents», p. 218; el de ReAct, Yao et al. (2023), §2, p. 3. El máximo de replanificaciones es [AGREGADO - no está en las fuentes]: es la regla de la rúbrica aplicada al mecanismo, y se proyecta como estimación del curso

Los tres patrones: qué compra, qué cuesta, cómo se protege

- ReAct — compra exploración anclada en hechos · cuesta 1 llamada por paso · se protege con detector de bucles y presupuesto
- Plan-Execute — compra coherencia y revisión del plan · cuesta rigidez, que mitiga replanificar · se protege con un tope de replanificaciones
- Reflexion — compra mejora con retroalimentación · cuesta ×2-4 llamadas (estimación del curso, no del paper) · se protege con un verificador REAL
- El jueves entran las tres filas que faltan: descomposición de tareas, paralelismo y hand-off entre agentes
- Chain of Thought, function calling y memoria episódica NO entran en esta tabla: no son patrones de control
- La Parte 5 del notebook clasifica una lista de diez patrones de divulgación y busca su duplicado

la tabla tiene criterio de pertenencia			
patrón	compra	cuesta	protección
ReAct	exploración anclada en hechos	1 llamada/paso	detector de loops + presupuesto
Plan-Execute	coherencia y revisión del plan	rigidez	replanning — abre un bucle: con tope
Reflexion	mejora con feedback	×2-4 llamadas (estimación del curso)	verificador REAL
sesión 14, jueves	Task Decomposition		
	Parallel Execution		
	Agent Handoff		

✗ Chain of Thought ✗ function calling ✗ memoria episódica

no son patrones de control: no entran en esta tabla

las tres filas vacías son de la sesión del jueves; el coste de Reflexion es estimación del curso, no del paper

Fuente: curso/actividades/_fuentes/s3-mar.py, líneas 268-278 (la tabla y la línea que impide el error de nivel); s3-mar-explicacion.html §4, líneas 86-93. El «×2-4» es del material del curso y no aparece en Shinn et al. (2023): se proyecta como estimación del curso. El «tope de replanificaciones» es [AGREGADO - no está en las fuentes] y se proyecta igual, como estimación del curso: ninguna fuente del repositorio fija ese máximo. La lista de diez es fuentes/web/material-divulgacion.md, ítem 2 — afirmación bajo análisis, no fuente

Diseño del toolset: la mitad que las lecturas no cubren

12

Cinco líneas que el lunes tenían y el sábado no

- El notebook del lunes envuelve la llamada a la tool y devuelve ERROR: {e} al contexto del modelo
- El andamiaje del Lab 03 no lo hace: la excepción sube por `_execute_action` → `run` → `__main__` y mata el proceso
- Y `_write_trace` solo se llama en `completed` y en `max_steps_reached`: la corrida que revienta no deja traza

```
# s3-lun.py 200-208 (lunes): la tool va envuelta
try:
    resultado = TOOLS[nombre](**entrada)
    error = False
except Exception as e:
    resultado, error = f"ERROR: {e}", True # vuelve al modelo

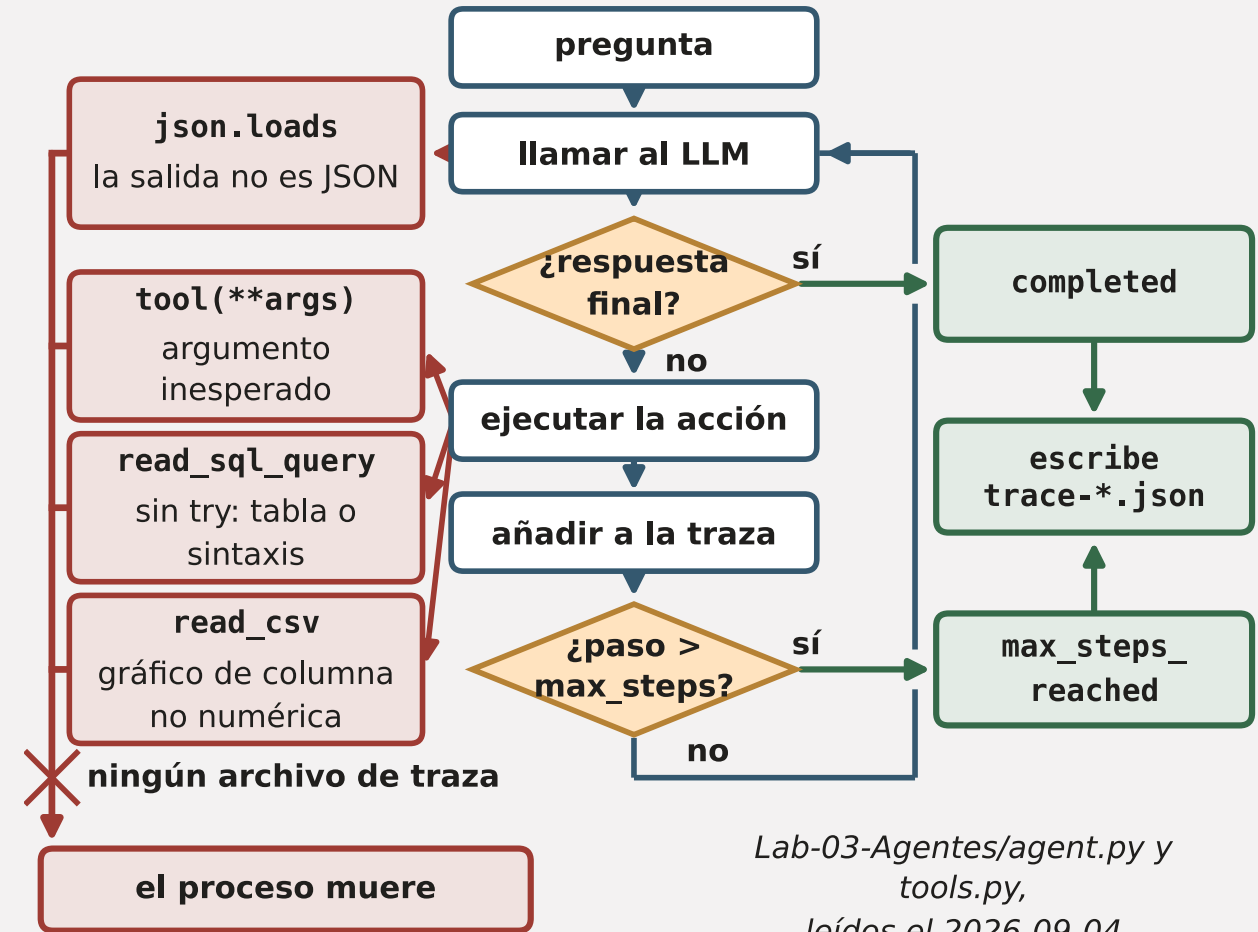
# agent.py 66-72 (Lab 03, lo que se califica el sabado)
tool = TOOL_REGISTRY.get(name)
if tool is None:
    return {"error": f"Tool no registrada: {name}"}
return tool(**args) # sin try y sin esquema
```

Fuente: `curso/actividades/_fuentes/s3-lun.py`, líneas 200-208; `curso/labs/Lab-03-Agentes/agent.py`, líneas 66-72 (`_execute_action`), 90 y 106 (las dos únicas llamadas a `_write_trace`) y 109-112. Los cinco archivos del laboratorio se leyeron completos el 2026-09-04

Seis huecos del andamiaje que se califica el sábado

- 1 · `tool(**args)` sin esquema: un nombre de argumento equivocado lanza `TypeError`, y nadie lo captura
- 2 · `pd.read_sql_query` sin `try/except`: una tabla inexistente o un error de sintaxis matan la corrida
- 3 · Ninguna tool describe el esquema de la base, y el enunciado exige mínimo 3 tablas: el agente adivina
- 4 · `row_limit` es un parámetro público de la tool: el límite de filas lo elige el modelo
- 5 · `generate_chart` recibe `output_path`: el toolset «de solo lectura» escribe donde el modelo diga
- 6 · `_call_llm` descarta `response.usage`: el presupuesto de tokens que pide el taller no es implementable sin tocarlo

dos salidas ordenadas, cuatro que matan el proceso



Lab-03-Agentes/agent.py y
tools.py,
leídos el 2026-09-04

El modelo escribe `SELECT * FROM ventas` y la tabla se llama sales

Es la primera ejecución del agente que van a entregar el sábado, con el andamiaje tal como está

(a) ¿Qué ve el modelo: un error que puede corregir, o nada?

(b) ¿Queda archivo de traza de esa corrida?

(c) ¿Cómo supo el modelo que la tabla se llamaba ventas?

(d) ¿Cuál es la herramienta que falta, y por qué el enunciado la vuelve obligatoria de hecho?

Contesten señalando las líneas que acaban de ver, no de memoria

El contrato mínimo de una herramienta — síntesis del curso

- El espacio de acciones acota el techo del agente: ReAct lo hizo débil a propósito y lo declaró
- Nombre y descripción escritos para el modelo; esquema de entrada tipado, con required y dominio cerrado donde lo haya
- Validar antes de actuar. El error se devuelve como valor y dice cómo corregirlo, no solo que falló
- La salida es acotada y su límite NO viaja en el esquema público: vive donde el modelo no llega
- Efectos secundarios declarados: qué lee, qué escribe, con qué permisos. Y entorno aislado si ejecuta código
- Idempotencia y observabilidad: ¿es seguro repetirla en un bucle?, ¿deja nombre, argumentos, resultado, latencia en ms y bandera de error?

seis cláusulas, y ningún código del curso las cumple todas

síntesis del curso — ninguna fuente reúne estas seis cláusulas

	el notebook del lunes	el Lab 03
nombre y descripción escritos para el modelo	sí	no
esquema de entrada tipado, con required y dominio cerrado	sí	no
validar antes de actuar; el error se devuelve como valor	parcial	sí
la salida es acotada por el sistema	no	parcial ¹
efectos secundarios declarados; entorno aislado si ejecuta código	no	no ²
idempotencia y observabilidad de la llamada	sí	parcial

¹ el límite de filas viaja en el esquema público: lo elige el modelo

² el docstring dice «solo lectura» y la función escribe un archivo

el Lab 03 registra 3 herramientas y ninguna descripción; verificado en el repositorio el 2026-09-04

Fuente: [AGREGADO - no está en las fuentes] como lista unificada: ninguna fuente del repositorio reúne estas seis cláusulas. Respaldo por cláusula: Yao et al. (2023) §3.1, p. 4; `_fuentes/s3-lun.py`, líneas 42 y 60–85; `Lab-03/tools.py`, 17, 22–25, 32–33 y 71; `curso/talleres/taller-03-agentes.md`, línea 56; Shinn et al. (2023) §8, p. 9; `fuentes/notas-teoricas/s3-agentes.md` §5.3

Mañana, cómo se descubre. El sábado, el agente

- Un pensamiento es una acción: por eso la traza del taller se escribe con Thought, Action y Observation
- Toda ganancia va con su benchmark y con su modelo base, o no es una ganancia
- Reflexion no supera a su verificador; sin señal verificable, reflexionar empeora el resultado
- El error se devuelve, no se lanza. Y un límite que viaja en el esquema público no es un límite
- Miércoles (13): MCP, cómo se DESCUBRE una herramienta. Jueves (14): varios agentes y sus riesgos
- Sábado: Taller 3, 25 % de la nota final, 3 de octubre, en grupos, sobre el mismo sistema del Taller 2